

第 5 章内存管理

地址翻译·换页与压缩·终端内存机制 (OOM / Hyperhold / Purgeable)

江西科技师范大学

万博洋

2026 年 10 月 10 日 (第 5 讲)

版权与来源说明

使用声明

本课件仅供教学使用，以《智能终端操作系统与 OpenHarmony 实践》（陈海波等，电子工业出版社）为基准编写。

- 教材插图：图 5-1、图 5-2 按图号标注，其余为课程自制图补足
- 论文数据：OSDI'24 官方 PDF (§4.5 / §6.2)，逐条标注出处
- 课程自制图标注“课程自制”；代码为示意代码，非鸿蒙源码
- 实验输出为模拟器实拍（API 24），命令逐条可复核

本讲目录

- **5.0 虚拟内存与地址空间**（虚拟地址 / 物理地址 / 页）
- **5.1 操作系统的内存管理基础**（地址翻译和页表 / 换页机制 / 虚拟内存压缩）
- **5.2 面向智能终端特征的内存管理机制**（OOM Killer / Hyperhold / Purgeable Memory）

每个部分配一个终端实验：在模拟器上把“看不见的内存”变成可观察的数据。

本讲导学

导学

手机 12GB 内存为什么还会卡？后台微信点开为什么像“重新启动”？本讲从“虚拟地址如何变成物理地址”讲到“内存不够怎么办”：翻译、换页、压缩，再到终端特有的 OOM Killer、Hyperhold 与 Purgeable Memory。

- 关键词：虚拟内存·页表·TLB·缺页异常·zRAM·OOM Killer·Hyperhold·Purgeable Memory
- 带着问题听课：4GB 物理内存的模拟器，凭什么让每个进程都“觉得自己独占一整块内存”

上节回顾

- 第 4 讲：处理器调度——决定“谁运行、在哪个核、以什么频率运行”
- 负载均衡把任务在核间迁移，但迁移有代价：缓存失效，**包括地址翻译缓存**（本讲主角 TLB 登场）
- EAS 高负载保性能、低负载省电；`sched_ext` 让策略可插拔——“机制与服务分离”贯穿始终

第 1 部分

5.0 虚拟内存与地址空间

5.0 定义：虚拟内存

定义（教材 5.1）

虚拟内存（Virtual Memory）是操作系统引入的一层抽象：把**有限且离散**的物理内存资源，高效地表示为**无限且连续**的虚拟内存。应用进程运行时使用**虚拟地址**（Virtual Address, VA）而非**物理地址**（Physical Address, PA）；CPU 内的**内存管理单元（MMU）**依据进程的**页表**（虚拟页 → 物理页的映射表）自动把 VA 翻译成 PA。

- 每个进程拥有**独立且连续**的虚拟地址空间——并发应用不会地址冲突
- 进程只能访问页表里配置的物理页——**隔离与共享**都由此实现

5.0 虚拟内存的三个“礼物”

- **① 隔离**：进程只能看见自己的地址空间——恶意程序摸不到别的进程和内核（配合用户/内核位，第 3 讲双世界）
- **② 连续**：应用假设自己的内存是连续的（malloc 一大块直接用）——物理内存却可以零散分布在任意角落
- **③ 按需**：用多少装多少——虚拟空间可以远大于物理内存（模拟器实测：全系统承诺 27GB，物理只有 4GB）

一句话：虚拟内存是“给应用的承诺”，物理内存是“实际兑现”——兑现方式就是本讲的翻译与换页。

5.0 虚拟地址、物理地址与页

- **页 (Page)**: 把虚拟地址空间切成**连续等长**的小块 (终端常用 **4KB**); 物理内存切成同样等长的**物理页 (页框)**
- 虚拟地址 = **虚拟页号** (第几页) + **页内偏移** (页内第几字节)
- 物理地址 = **物理页框号** + 同一个页内偏移——偏移不用翻译, 直接抄过去
- 页表只需记录 “虚拟页号 → 物理页框号” ——粒度是**页**, 不是字节

类比: 酒店房号 = 楼层房间号 (页号) + 门内位置 (偏移); 搬家只换房间号, 屋里的布局不变。

5.0 进程地址空间布局

- 一段一段划分：**代码段**（只读可执行）、**数据段**、**堆**（向上长）、**共享库**（mmap 映射区）、**栈**（向下长）——每一段是一个 **VMA**（Virtual Memory Area，虚拟内存区域，内核管理地址空间的基本单位）
- 高地址段留给**内核空间**（第 3 讲双世界：用户态不可访问）；低地址段是**用户空间**
- ARM 的分工：**TTBR0** 持用户空间页表基址，**TTBR1** 持内核空间页表基址（按高 16 位——第 63~48 位——全 0 / 全 1 选择）

布局是“账本”：缺页处理第一步就是查 VMA 判断地址合不合法（5.1.2）。

5.0 随堂练习：拆地址

例题示范（4KB 页·48 位虚拟地址·四级页表）

拆 $VA = 0x0000_1234_5678$ ：偏移 = 低 12 位 = $0x678$ ；L3 索引 = 第 20~12 位 = $(0x12345678 \gg 12) \& 0x1FF = 325$ ；L2 = 145；L1 = 0；L0 = 0——低 1GB 区间，高两级索引全是 0。

- ① 口算： $VA = 0x0000_00AB_CDEF$ 的页内偏移是多少？
- ② 上题地址的 L3 索引是多少？（提示： $(0xABCDEF \gg 12) \& 0x1FF$ ）
- ③ 口答：为什么页内偏移“不用翻译、直接抄”？

答案（先做再看）：① $0xDEF$ ② $0x0BC = 188$ ③ 偏移在页内的相对位置不因映射改变——翻译只换页号。

实验 5.0：看见地址空间

先在 PC 终端连接三步，再进 hdc shell 执行

- 连接：hdc list targets (空则启动模拟器) → hdc tconn 127.0.0.1:5555 → hdc shell
- cat /proc/meminfo | head -3: MemTotal 约 4GB、MemAvailable 约 2.5GB——物理内存家底；再 grep -i commit: **Committed_AS 约 27GB**——全系统承诺的虚拟内存
- cat /proc/1/status | grep -E 'VmSize|VmRSS': init 的 **VmSize 37MB vs VmRSS 7.7MB**——虚拟地址空间 $\neq$ 物理驻留 (按需分配)
- ps -e -o pid,vsz,comm | head -4: 内核线程 VSZ = 0——它们没有用户地址空间

第 2 部分

5.1 操作系统的内存管理基础：翻译·换页·压缩

5.1.1 分页机制：页表是映射表

定义（教材 5.1.1）

分页机制把虚拟地址空间划分为连续等长的**虚拟页**，把物理内存划分为连续等长的**物理页**；页表（Page Table）即**虚拟页到物理页的映射关系表**。虚拟页可以映射到任意物理页——物理内存得以**离散分配**。

- 翻译单位是页：页内偏移不变，页号查表替换
- 谁来查表？**硬件查**：MMU 每条访存指令自动完成——应用与内核都无感

5.1.1 AArch64 四级页表与翻译流程

- 常见设置（教材）：虚拟地址**低 48 位**参与翻译，**4 级页表**，页大小 **4KB**（低 12 位 = 页内偏移）
- 每级索引 **9 位**：47 ~ 39 → L0、38 ~ 30 → L1、29 ~ 21 → L2、20 ~ 12 → L3
- 每个页表项（PTE）**8 字节**：物理页框号 + 权限位（读写执行）+ 用户/内核位 + **有效位**（无效 → 触发缺页异常，5.1.2 详述）
- 每个页表页恰占一页： $2^{12}/8 = 512$ 项——所以索引是 9 位（ $2^9 = 512$ ）
- 走查起点：TTBR 寄存器给出 L0 页表的物理地址，MMU 逐级取表项

记法：48 = 12（偏移）+ 4×9（四级索引）。

5.1.1 为什么要分级：单级页表的爆炸

表: 单级页表 vs 四级页表 (48 位地址 / 4KB 页)

	单级页表	四级页表
表项数量	2^{36} (虚拟页数)	用到哪级建哪级
表项内存	$2^{36} \times 8\text{B} = 512\text{GB}$ /进程	常用区域只建少量中间表
适用性	比物理内存还大——不可行	AArch64 标准方案

- 关键思想：**中间级页表按需分配**——没用到的地址区间，中间表根本不存在
- 代价：一次翻译最多查 4 次页表——下次访存的延迟被放大 (→ TLB, 转址旁路缓存, 下页详述)

5.1.1 TLB：给翻译加一层缓存

- **TLB** (Translation Lookaside Buffer, 转址旁路缓存)：CPU 内缓存**最近使用的虚拟页** → **物理页映射**
- 命中：跳过四级走查，翻译近乎免费——程序的**局部性**（刚访问过的页马上还会访问）保证命中率很高
- 未命中：才走四级页表走查，再写入 TLB
- **大页**（教材小知识）：ARM 支持 16KB/64KB 基础页面，还支持在 L1/L2 表项直接存物理页地址——**L1 表项直达 1GB 大页、L2 表项直达 2MB 大页**——TLB 项更少；代价是**页内碎片**。Android 15 起支持 16KB 页面配置

与第 4 讲呼应：负载迁移导致 TLB 失效——迁移不是免费的。

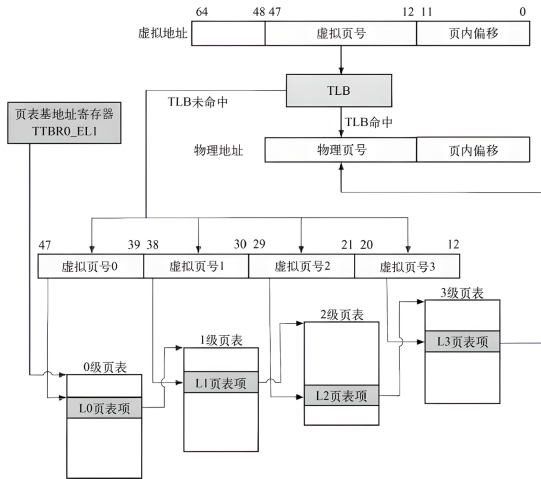
5.1.1 随堂练习：页表算术

- ① 页大小改用 **64KB**、仍是 48 位地址四级页表：偏移几位？每级索引几位？
- ② 经典复刻：**32 位**地址、4KB 页、**单级**页表、每项 4 字节——每个进程的页表多大？
- ③ 口答：大页为什么能减少 TLB miss？代价是什么？

答案（先做再看）

① 偏移 16 位， $(48 - 16)/4 =$ 每级 **8 位** ($2^8 = 256$ 项/页表页)；② 2^{20} 项 $\times 4B =$ **4MB/进程**——100 个进程仅页表就 400MB，这就是“分级”的 32 位版动机；③ 大页让同一空间占更少 TLB 项——代价是页内碎片（分配粒度变大）。

5.1 四级页表地址翻译（教材图 5-1）



5.1.1 代码示意：页表项与四级走查

```
1  typedef struct {                                /* 页表项 PTE: 8 字节 */
2      unsigned long pfn    : 36;                 /* 物理页框号 (4KB 页) */
3      unsigned long valid: 1;                     /* 有效位: 0 -> 触发缺页异常 */
4      unsigned long user  : 1;                     /* 用户/内核位 */
5      unsigned long rw    : 1;                     /* 写权限位 */
6  } pte_t;
7  #define IDX(va, lv) (((va) >> (39 - 9*(lv))) & 0x1FF) /* 各级 9 位索引 */
8  #define OFF(va)    ((va) & 0xFFF)                 /* 11~0 位: 页内偏移 */
9  void *walk(unsigned long va, pte_t *l0) {
10     pte_t *e = &l0[IDX(va, 0)];                 /* L0 表项 (TTBR0 起点) */
11     for (int lv = 1; lv < 4 && e->valid; lv++)
12         e = &table_of(e)[IDX(va, lv)]; /* 逐级: L1 -> L2 -> L3 */
13     return e->valid ? page_of(e->pfn) + OFF(va) : 0; /* 无效 = 缺页异常 */
14 }
```

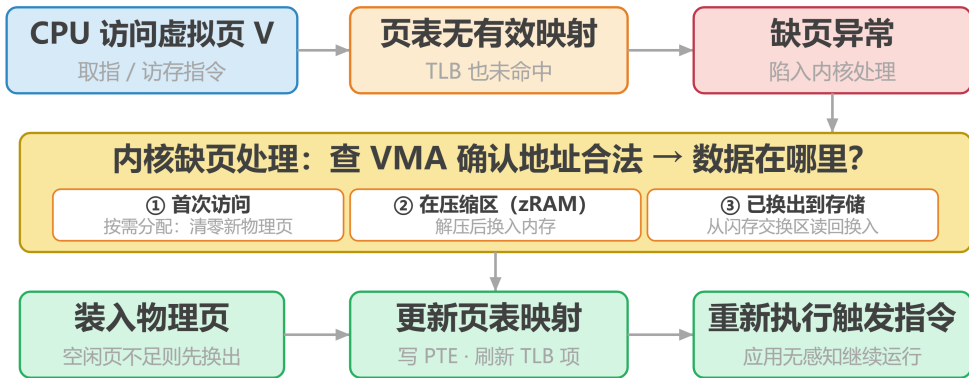
5.1.2 定义：换页机制

定义（教材 5.1.2）

换页机制（Page Swapping）：物理内存不足时，把物理页的内容写入**容量更大、成本更低的存储设备**，释放物理页继续使用——不修改应用，即获得超过物理容量的内存空间。

- **换出**：写存储 → 移除页表映射 → 记录存储位置 → 物理页另作他用
- **换入**：再次访问触发**缺页异常** → 确认地址合法 → 分配空闲页 → 读回 → 更新页表 → 断点继续
- 终端的换出目标通常不是磁盘，而是**内存压缩区（zRAM，内存中的压缩块设备）或闪存交换区**

5.1 缺页异常与换页闭环（课程自制）



换出 = 冷页写入存储释放物理页；换入 = 缺页时从存储读回（教材 5.1.2）

5.1.2 淘汰谁：局部性、LRU 与颠簸

- 换页的前提是“选页淘汰”——依据是**局部性原理**：刚访问过的数据，马上再被访问的概率高（时间局部性 / 相邻地址也常被访问：空间局部性）
- **LRU** (Least Recently Used, 最久未使用)：淘汰最长时间没被访问的页——理想但精确维护太贵；实际用**时钟 (Clock) 算法**等近似
- 内核把页挂在 **Active / Inactive** (活跃/不活跃) 两类 LRU 链上（再按 anon / file 各分一支，见实验输出）分冷热——冷页优先回收
- **工作集** (Working Set)：进程近期实际访问的页的集合；物理内存装不下工作集时，刚换出的页马上又要换入——**颠簸 (Thrashing)**：I/O 打满、吞吐骤降

LRU/Clock/工作集为经典理论，教材未展开——实验里用 Active/Inactive 双链表验证“分冷热”。

5.1.2 随堂练习：淘汰手算

题目 (3 帧 vs 4 帧，内存初始为空，淘汰策略 FIFO——先进先出，淘汰最早进入的页)

访问序列：1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 ——各缺页几次？

- 第一步：列表逐项模拟（帧内容 + 是否命中）；第二步：数 miss
- 陷阱提示：帧数**多**不一定缺页**少**——先做完再对答案

答案（先做再看）

FIFO：3 帧 **9** 次，4 帧 **10** 次——帧数增加缺页反而变多，这就是 **Belady 异常** (FIFO 特有)。课堂延伸：同一序列用 LRU 重算——3 帧 10 次、4 帧 8 次，**无异常**：LRU 是“栈算法”，帧多的策略覆盖帧少的策略，缺页数单调不增。

实验 5.1：看见换页与冷热

在 hdc shell 中执行

- `cat /proc/meminfo | grep -iE 'swap|active|inactive':`
SwapTotal 约 2GB、已用约 170MB——换出空间真实存在；
Active/Inactive 冷热链表组——页分冷热
- `cat /proc/1/stat:` 第 **10** 字段 = 次要缺页 (minflt, 13176: 按需分配, 免 I/O); 第 **12** 字段 = 主要缺页 (majflt, 392: 需从存储读回)
- 对照理解: minflt 多而 majflt 少 $\Rightarrow$ 局部性好、换页开销可控

观察点: ① 2GB swap 就是“超卖”的额度 ② Active/Inactive 冷热链表组是 LRU 的工程形态 ③ majflt 每次都要读存储, 单次代价比 minflt 高得多 (次数 13176 vs 392)。

5.1.3 定义：虚拟内存压缩

定义（教材 5.1.3）

虚拟内存压缩机制：内存资源紧张时，选择**近期不常用的内存页进行压缩**，而不是直接写入存储设备，从而释放更多空闲内存；需要访问时再解压。**所有操作都在内存中完成**——比换出到存储更快释放、更快恢复。

- 本质：用 **CPU 时间**换 **I/O 时间**——压缩/解压耗算力，但省下慢得多的存储 I/O

5.1.3 zswap：压缩与换页的结合

- Linux 的 **zswap**：在内存中开辟**压缩缓冲区**——准备换出的页先压缩放入 zswap 区域
- 好处：**延迟写盘**——攒够一批再批量 I/O，甚至**完全避免 I/O**；最终真要写盘时，数据量也已显著变小
- 再访问：把压缩数据**解压换入**内存

两个前提要记住（教材原文）

① zswap 仍消耗物理内存——扩展能力**有限**；② 若回收不精准，频繁压缩/解压会**占用 CPU、增加功耗**，甚至仍引发大量 I/O——策略不当时两头吃亏。

zswap = 压缩缓存，zRAM = 压缩块设备——Hyperhold 要解决的正是“压得准”（5.2）。

5.1.3 随堂练习：压缩算术

- ① 计算：zRAM 分区 **1GB**、平均压缩比 **3:1** (4KB 页压到约 1.33KB)——能容纳多少原页数据？相当于把内存“扩大”了多少？
- ② 口答：什么情况下压缩“不划算”，反而两头吃亏？
- ③ 口答：同样是回收 1GB，压缩与换出到 UFS 各付出什么代价？

答案（先做再看）

① $1\text{GB} \div (4\text{KB}/3) \approx$ 存 **3GB** 原页数据——1GB 分区当 3GB 用，净赚 2GB；② 页刚被压缩就又被访问 (refault 高)：白付解压的 CPU 时间，还可能被换出后再换入——教材 5.1.3 的“两头吃亏”；③ 压缩付 **CPU 与内存** (压缩区本身占内存)、换出付 **I/O 时间与闪存寿命**——Hyperhold 按 refault 在两者间选路。

第 3 部分

5.2 面向智能终端特征的内存管理机制

OOM Killer (杀谁) · Hyperhold (怎么扩) · Purgeable Memory (丢什么)

5.2 定义：终端内存的三个难题

三个难题与三个机制（据教材 5.2 归纳）

终端内存容量小、任务多、切换频繁，通用机制不够用。

OpenHarmony 用三个机制分别回答三个问题：**无内存可用怎么办（OOM Killer）、换页太慢怎么办（Hyperhold）、缓存占用浪费怎么办（Purgeable Memory）。**

- 场景（教材开篇）：前台游戏要内存申请失败退出；系统杀掉后台微信回收内存——用户再点开微信就要忍受重新初始化的时延
- 内存管理的目标从来不是“全保住”，而是**该保的保住、该舍的先舍**

5.2.1 OOM Killer: 目标与触发

定义 (教材 5.2.1)

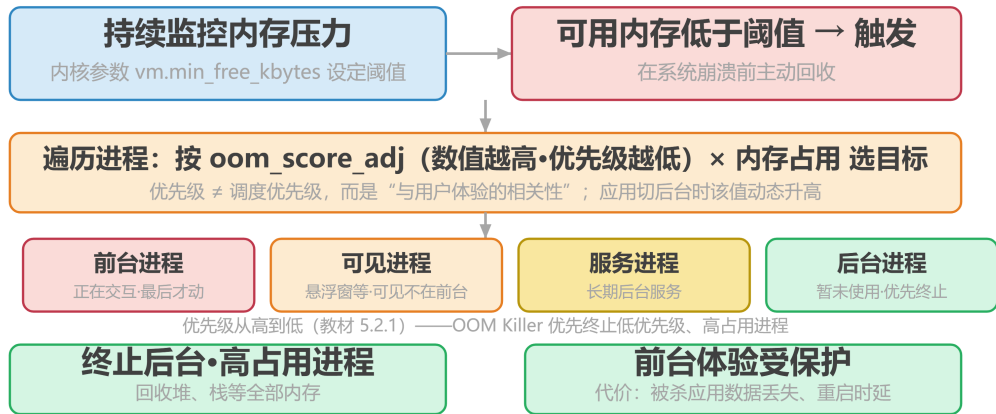
OOM (Out-Of-Memory) Killer: 内存资源紧张时, **有策略地终止进程**来释放内存, 确保系统稳定与流畅体验。

- 两大目标: ① **防止系统崩溃**——内存耗尽前主动回收; ② **优化用户体验**——优先保留用户正在用的应用
- **触发**: 内核持续监控内存, 低于阈值 (内核参数 `vm.min_free_kbytes`) 时触发
- 小知识: OOM Killer 是内核最后手段, 调用时已很危险; Android 的 LMK (Low Memory Killer) 借框架层更早介入——优先杀空闲应用

5.2.1 选择原则：oom_score_adj

- 遍历进程列表，按 oom_score_adj（数值越高优先级越低）× 内存占用选目标——注意这**不是调度优先级**，是“与用户体验的相关性”
- 四类进程（保护力度递减）：**前台**（正在交互）> **可见**（悬浮窗等）> **服务**（后台长驻）> **后台**（暂未使用，优先终止）
- 该值**动态变化**：应用在前台时数值低（优先级高），切到后台即升高（优先级降低）——Android 由 ActivityManagerService 等动态调整
- 代价：被杀应用**数据丢失**、重开时延——所以是最后防线，前面还有压缩与换页

5.2 OOM Killer 决策流程（课程自制）



5.2.1 随堂练习：谁先被杀

情景：内存紧张，OOM Killer 启动——

进程	oom_score_adj	RSS
A 前台游戏	0	1.2GB
B 可见悬浮窗	100	80MB
C 后台微信	500	600MB
D 后台下载器	700	300MB

❶ 先杀谁？为什么不是占用最大的 A？ ❷ A 切后台、adj 升到 600，再杀谁？

答案（先做再看）：❶ 杀 D——adj 最高（优先级最低）且高占用；A 因 $\text{adj} = 0$ 受最强保护。❷ 杀 A—— $\text{adj} 600 > 500$ 且占用最大；“切了后台的霸主也逃不过”，adj 动态调整是核心。

5.2.2 为什么手机不做传统 swap

- 传统 swap（整页换到存储）在手机上两大死穴：
 - 存储带宽低，换入换出遭遇 **I/O 性能瓶颈**
 - 按页面级冷热交换——**同一应用的页面被打散**到存储各处，产生严重**随机 I/O**（比顺序 I/O 劣化严重）
- 多年实践（教材原话）：手机**不将内存交换到存储器件**，改用**压缩内存模拟存储后端**——即 zswap/zram 技术；但它只能**有限扩展**
- 要“近似无限”的扩展，必须**基于存储器件**——落地关键就是把 I/O 瓶颈解决好

硬件前提：换页能不能上闪存，取决于存储接口的速度——下一页。

5.2.2 eMMC 与 UFS：换页的硬件前提

表: eMMC（嵌入式多媒体卡）与 UFS（通用闪存存储）对比（教材小知识）

	eMMC	UFS
传输带宽	通常 400MBps 以下	高约 1 个数量级
随机小 I/O 时延	400~500 μ s	低约 1 个数量级
换页策略	只做内存压缩， 不做换页 （时延抖动不可接受）	为换页提供硬件支撑

- 结论：Hyperhold 的“聚合换出”建立在 **UFS** 之上；教材口径——eMMC 设备**通常只做内存压缩、不做换页**
- 与 W4 呼应：存储快了，才敢把“换页”加进资源调度的大棋局

5.2.2 定义：Hyperhold 内存扩展

定义（教材 5.2.2）

Hyperhold：OpenHarmony 提出的内存管理机制，**综合考虑应用数据冷热特征、内存压力、存储设备寿命、硬件算力**，支持**高性能内存压缩与非关键数据聚合换出**——实现“应用生命周期的内存管理”。

- 架构：**框架层服务 + 内核层 ZRAM 与 ESWAP (Enhanced SWAP) 模块**——传统 zswap 只有内核层，**缺失应用运行时语义**
- 框架层懂应用（生命周期、交互习惯），把**内存语义告知内核** → 按关联性分组、精准回收

5.2 Hyperhold 框架（教材图 5-2）

Hyperhold 内存管理框架



5.2.2 Hyperhold 怎么干活

- 按 Linux **memcg**（内存控制组）分组，不同分组配置不同回收数量——用**回收优先级**区分活跃度，优先压缩、换出**不活跃**数据
- **zswapd 守护线程**（类比 Linux kswapd）：定期监控，按 **refault**（被压缩/换出回收的页又被访问、需重新换入——说明压错了人）频率决策——
 - refault 不高 → 继续留在 ZRAM 压缩（沿 memcg 的 LRU 链选页释放）
 - refault 比例超水线 → 触发 **ESWAP**：遍历 memcg **聚合换出**到存储，达到目标量即停
- 匿名页（栈/堆等没有文件对应的内存）三个去向——**内存 / ZRAM / ESWAP**，比例可灵活配置，兼顾空间、体验、寿命

实测：模拟器里 zswapd0 与 kswapd0 并存——回收主力已换人。

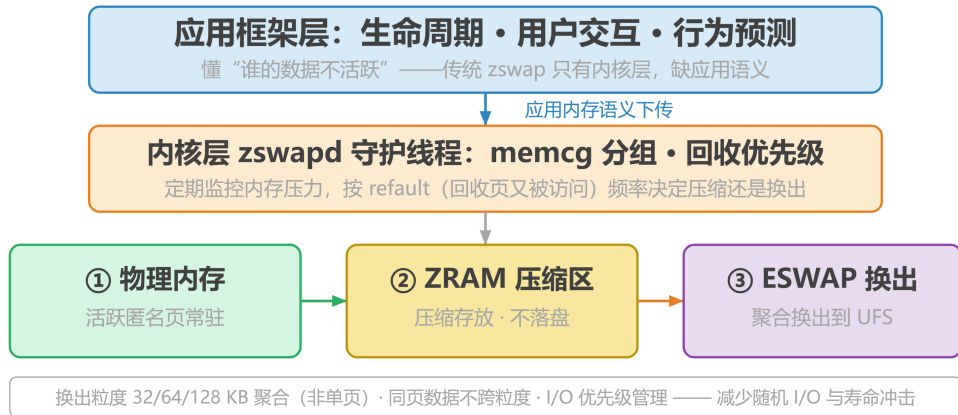
实验 5.2：看见压缩线程与 Purgeable

在 hdc shell 中执行

- `ps -ef | grep -E 'zswapd|kswapd' | grep -v grep`: **zswapd0 在岗** (另有 kswapd0) ——回收主力是 Hyperhold 线程
- `hidumper --mem | head -4`: Pss/Vss/Rss/Uss 列, 最右 **PurgSum/PurgPin** 列——Purgeable 一等公民
- `ps -e -o pid,rss,comm | sort -k2 -n | tail -3`: 内存大户 (ohos.sceneboard 约 700MB) ——OOM 候选视角

观察点: ❶ 回收主力已从 kswapd 换成 zswapd ❷ Purg 列就是 5.2.3 的标记内存 ❸ RSS 排序即“杀谁”的候选清单。

5.2 Hyperhold 三级去向（课程自制）



5.2.3 Purgeable: 可清除的缓存

定义 (教材 5.2.3)

Purgeable Memory: 允许应用把**可轻松再生成**的内存区域（缓存、临时数据）**标记为可清除**；内存紧张时系统**优先回收**这些区域，**无需通知原应用**直接释放。

- 场景：网页图片、视频解码、游戏缓存——**匿名页**里的“可再生”数据
- 三步实现：① **标记** ② **优先回收**（不通知直接释放） ③ **再生成**（用时重建）
- 收益：省内存、降功耗、避免崩溃

实测：hidumper --mem 输出表自带 PurgSum/PurgPin 列——Purgeable 是一等公民。

5.2 三机制对比：一张表看清分工

表: 终端内存三机制对比

机制	回答的问题	工作方式	代价
OOM Killer	彻底没内存了，杀谁	按 oom_score_adj 终止低 优先级高占用进程	数据丢失 / 重启时延
Hyperhold	换页太慢，怎么扩	压缩 (ZRAM) + 聚合换出 (ESWAP)	CPU / 存储寿命
Purgeable	缓存占用浪费，丢什么	丢弃可再生缓存，用时重 建	重建计算 / 重载 I/O

回收层次（课程归纳，教材未给此排序）：可清除先丢 → 冷页压缩 → 聚合换出 → 实在不行才杀进程——代价从低到高。

课堂思考：为什么终端宁愿杀进程，也不愿长时间换页？

先独立思考，再对照解答（下一页）

结合 5.1 的换页代价与 5.2 的机制分工来推理——注意题目问的是“长时间换页”。

解答：换页的代价阶梯

参考解答

- 换页的瓶颈是**存储 I/O**：长时间大量换页会占满 I/O 与 CPU，前台每一次访存都可能排队等换入——卡顿立即被用户感知
- 杀后台进程的代价是“**下次打开慢**”（重新初始化时延，如教材开篇的微信）——当前前台完全不受影响
- 所以顺序是：先丢可再生缓存（Purgeable）→ 压缩（ZRAM）→ 聚合换出（ESWAP）→ 最后才 OOM

换页的痛是“现在痛且前台痛”，杀进程的痛是“以后痛且后台痛”。

AI 案例：无策略内核分页

- 微内核两难：为保“最小内核”，内存管理器放在内核外（集中式，cgroup/回收需要全局视图）——但每次缺页要**内核** → **pager** → **内核**的 IPC 往返，页缓存缺失处理占启动 CPU 的 16%，比 Linux 慢 2 倍（论文图 2）
- **无策略内核分页**（OSDI'24 论文 §4.5，第 3 讲同款）：**策略前置**——内存管理器预先划定匿名内存区间并预分配物理页；核心内核保留“无策略”的快速缺页处理：查地址 → 直接映射预分配页 → 记 OPLog（内存管理器异步对账）
- 实测（论文图 8）：缺页延迟低于 Linux（Kirin 写 1816 vs 1921 ns）；整机启动快 17%、切换快 32% (§6.2)

Take-away (本讲小结)

核心要点

- 虚拟内存三礼物：隔离、连续、按需； $VA = \text{页号} + \text{偏移}$ ，MMU 查页表翻译
- AArch64 四级页表：48 位 = 12 偏移 + 4×9 索引；分级省内存；TLB 缓存翻译、大页减 miss
- 换页：换出/换入经缺页异常；LRU 与工作集；压缩 = 用 CPU 换 I/O
- 三机制分工：OOM 杀后台保前台 / Hyperhold 压缩 + 聚合换出 / Purgeable 丢缓存
- 考点：四级页表位段、缺页流程、OOM 原则、三机制代价层次（归纳）

第 5 周实验卡：看见内存

本周必做 (30-60 分钟)

- 连上模拟器执行实验 5.0~5.2 命令，挑一个输出解释它印证的概念 (如 $VmSize \gg VmRSS \leftrightarrow$ 按需分配)

AI 验证题 (课后 3 题，用课件判据核对)

- ① 四级页表为什么比单级省内存？(核对：中间表按需分配 + 512 项/页)
- ② zRAM 为什么比写盘 swap 快？(核对：内存带宽 vs 闪存 I/O；代价是 CPU)
- ③ 为什么按 oom_score_adj 挑目标？(核对：保前台)

第 5 周实验卡：实测命令与输出

```
[1] 5.0 地址空间 | 按需分配
> hdc shell "cat /proc/meminfo | head -3"
MemTotal: 4025360 kB MemFree: 1704196 kB
MemAvailable: 2552280 kB
> hdc shell "cat /proc/1/status | grep -E 'VmPeak|VmSize|VmRSS'"
VmSize: 37872 kB <- init 虚拟地址空间 37MB
VmRSS: 7848 kB <- 物理驻留仅 7.7MB (按需分配)
> hdc shell "ps -e -o pid,vsz,comm | head -4"
PID VSZ COMM
1 37872 [init]
2 0 [kthreadd] <- 内核线程无用户地址空间

[2] 5.1 换页空间 | 冷热链表 | 缺页计数
> hdc shell "cat /proc/meminfo | grep -iE 'swap|active|inactive'"
Active: 378640 kB Inactive: 1572356 kB <- 冷热双链表
SwapTotal: 2097148 kB SwapFree: 1922812 kB <- 2GB 换出空间·已用约 170MB
> hdc shell "cat /proc/1/stat" # 第10字段 minflt 第12字段 majflt
1 (init) S 0 0 0 0 -1 4194560 [13176] 98683 [392] 1237 ...
^13176=次要缺页(按需分配·免I/O) ^392=主要缺页(需从存储读回)

[3] 5.2 守护线程 | Purgeable 列 | 内存大户
> hdc shell "ps -ef | grep -E 'zswapd|kswapd' | grep -v grep"
root 68 2 0 13:57:58 ? 00:00:00 [kswapd0]
root 70 2 0 13:57:58 ? 00:00:00 [zswapd0] <- Hyperhold 回收线程
> hdc shell "hidumper --mem | head -4"
PID Total Pss(xxx in SwapPss) ... PurgSum PurgPin Name
1 3823(0 in SwapPss) kB ... 0 kB 0 kB init
> hdc shell "ps -e -o pid,rss,comm | sort -k2 -n | tail -2"
2620 243284 [pGalleryService] 1524 701000 [ohos.sceneboard]
```

- ① init 虚拟空间 37MB vs 物理驻留 7.7MB (按需分配) ② 2GB 换出空间与冷热链表组 ③ zswapd0 在岗、Purgeable 列在册

延伸资源与实践

论文与文档

- **教材第 5 章**：地址翻译和页表 / 换页机制 / 虚拟内存压缩 / OOM Killer / Hyperhold / Purgeable Memory
- **OSDI'24 论文 §4.5 “无策略内核分页”**：官方 PDF 可 <https://www.usenix.org/system/files/osdi24-chen-haibo.pdf>
- **OpenHarmony 内存管理文档**：<http://docs.openharmony.cn> (Hyperhold、Purgeable)

延伸资源与实践（续 1）

源码入口与动手

- **Linux 源码阅读入口：** `kernel/mm/`（缺页处理、LRU）、`drivers/block/zram/`（zRAM 实现）
- **动手：** 把实验 5.1 里 `majflt/minflt` 的比值记下来，课后跑一轮大应用再对比——观察主要缺页的变化

作业与安排

本周任务

第 5 周作业《HarmonyMemLab》(3–5 小时, 见课程主页): 任务 A 实验 5.0~5.2 实测解读 30 分 + 任务 B Agent 结对三连问 25 分 + 任务 C 计算四道与走查图 30 分 + 任务 D 阅读思考 10 分 + 记录规范 5 分; **假期作业 (HarmonySchedLab) 随堂检查。**

下讲安排

10 月 14 日 (周三) 第 6 讲: **存储与文件系统**——预习: 教材第 6 章 “闪存的特点” 与 “F2FS” 两节。

下次课预告

预告

第 6 讲：存储与文件系统——数据如何落盘：闪存的擦写约束、F2FS 与 EROFS 如何为终端而设计

上课时间：**10 月 14 日 (周三)**；本讲实验卡随堂答疑